

LLM-Aided Testbench Generation — Report

Course: LLM4ChipDesign · **Student:** Tanvir Shahriar

Runs

	Module	Part
Tutorial example 1	mux2to1 — 2-to-1 multiplexer	I
Tutorial example 2	adder4bit — 4-bit adder	I
Custom module	sat_adder8 — 8-bit unsigned saturating adder	III

LLM setup

- **Provider / endpoint:** NVIDIA NIM, <https://integrate.api.nvidia.com/v1> (OpenAI-compatible, so only `base_url` changed in `LLMClient`)
- **Model:** `nvidia/nemotron-3-super-120b-a12b`
- **Parameters:** temperatures left as provided (0.7 for Steps 3 and 5, 0.2 for Step 4). Step 3's `max_tokens` was raised from 4000 to 12000.

Three prompt/parser issues had to be fixed to get the pipeline working with this model:

1. **Pattern count.** The adder description does not state how many patterns to generate. GPT-4o picks about 10; this model enumerated all 256 input combinations and the response was truncated. Fixed by appending an explicit count and forbidding for-loops.
2. **Token limit.** Step 3's 4000-token cap truncated the response before the JSON block closed, so the pattern list was lost.
3. **Marker mismatch.** The parser looks for the exact text `TEST_PATTERNS:`, but the prompt asks for it followed by a long description in brackets. This model copies that whole bracketed description into its answer, so the exact text the parser needs never appears and no patterns were found. Fixed by telling the model to write the marker on its own.

DUT interfaces and commands

mux2to1

Port	Dir	Width
a, b, sel	input	1 each
y	output	1

```
iverilog -g2012 -o output/mux/sim.vvp output/mux2to1.v output/mux/testbench_final.v
vvp output/mux/sim.vvp
```

Result: 8 patterns (exhaustive), 8 checks, 8 passed, 0 failed.

adder4bit

Port	Dir	Width
a, b	input	4 each
sum	output	4
carry	output	1

```
iverilog -g2012 -o output/adder/sim.vvp output/adder4bit.v output/adder/testbench_final.v
vvp output/adder/sim.vvp
```

Result: 20 patterns, 40 checks, 40 passed, 0 failed.

sat_adder8 (custom)

Port	Dir	Width	Meaning
a, b	input	8 each	unsigned operands
sum	output	8	a + b, clamped to 255
sat	output	1	high when a + b exceeded 255

```
iverilog -g2012 -o output/custom/sim.vvp output/sat_adder8.v output/custom/testbench_final.v
vvp output/custom/sim.vvp
```

Result: 20 patterns, 40 checks, 40 passed, 0 failed. Pattern coverage: 7 saturating, 13 non-saturating, 9 non-corner (random), with 6 patterns at `a+b == 255` and 3 at `a+b == 256`.

Part II — generated artifacts (adder)

golden_model.py — `adder4bit_golden(a, b)` takes two integers and returns `{'sum': ..., 'carry': ...}`, a dictionary keyed by output name. It adds the operands, masks the low four bits for `sum`, and sets `carry` when the total exceeds 15. The strict `>` means `7 + 8 = 15` gives `carry = 0`.

test_patterns_with_golden.json — a JSON array, one object per test, 20 in this run. Input names are top-level keys (`"a": 15`, `"b": 0`) and `expected_outputs` holds the expected value per output (`{"sum": 15, "carry": 0}`). The expected values come from running the Python

golden model.

Note (difference between LLMs). In this run Step 5 computed the expected values in Verilog (`{carry_exp, sum_exp} = a + b;`) instead of inlining the golden outputs from `test_patterns_with_golden.json`. The testbench passes, but its expected values no longer come from the Python reference. The `sat_adder8` run did not have this problem — its expected values are literals.

Part IV — bug detection

Injected bug in `sat_adder8`: on overflow the sum wraps instead of clamping.

```
assign sum = raw[8] ? 8'hFF : raw[7:0]; // correct
assign sum = raw[7:0];                // buggy
```

Failing run against the unchanged generated testbench — 7 of 40 checks failed, all on `sum`, all on saturating patterns, with every `sat` check still passing:

```
=== failing checks only ===
Test 4: a=11111111 b=11111111
      x sum=11111110 (expected 11111111)
Test 7: a=10000000 b=10000000
      x sum=00000000 (expected 11111111)
Test 8: a=11111111 b=00000001
      x sum=00000000 (expected 11111111)
Test 13: a=10010110 b=01101110
        x sum=00000100 (expected 11111111)
Test 14: a=11001000 b=01100100
        x sum=00101100 (expected 11111111)
Test 16: a=00001010 b=11111010
        x sum=00000100 (expected 11111111)
Test 19: a=01000000 b=11000000
        x sum=00000000 (expected 11111111)
```

After restoring the clamp, the same testbench and the same command pass:

```
checks passed: 40   checks failed: 0
```